



Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

ITEC 204 – Data Structures and Algorithms

Name: Joya, Julian Christopher J.	Date: September 24, 2026
Section: BSIT-2A	Instructor: Maxyne Nuela Ignacio

- I. **Objective:** Write a short explanation of what your program is designed to do. (Minimum of 3 sentences)

Answer: The objective of my program is to create an Incident Ticket Management System for organizing incident tickets. It uses a linear data structure to add, display, search, remove, and count incident tickets. This system helps manage active incidents in a simple and organized way.

- II. **Program Operations:** Briefly explain how your program performs each operation. (Insert Specific Code Line and Your Explanation)

Add Incident:

```
def add_ticket(self, incident_id, bot, description):
    # Check if a ticket with this ID already exists before adding
    if self.search_ticket(incident_id, verbose=False):
        print(f"[!] Ticket {incident_id} already exists. Not added.\n")
        return False

    new_node = Node(incident_id, bot, description)

    if self.head is None:
        self.head = new_node
    else:
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node

    self.size += 1
    print(f"[+] Ticket {incident_id} added successfully.\n")
    return True
```

Explanation: The program first uses `search_ticket()` to check if the Incident ID already exists. If it does not exist, a new node is created. If the list is empty, the new node becomes the head. Otherwise, the program goes through the list and adds the new node at the end. The size is then increased by 1.



Display Incidents:

```
def display_tickets(self):
    if self.head is None:
        print("[i] No active incident tickets.\n")
        return

    print("\n" + "-" * 70)
    print(f"{'Incident ID':<15}{'Bot':<20}{'Short Description'}")
    print("-" * 70)
    current = self.head
    while current is not None:
        print(f"{current.incident_id:<15}{current.bot:<20}{current.description}")
        current = current.next
    print("-" * 70 + "\n")
```

Explanation: If the list is empty, the program displays a message. Otherwise, it starts at the head and goes through each node one by one, displaying the Incident ID, Bot, and Description.

Search Incident:

```
def search_ticket(self, incident_id, verbose=True):
    current = self.head
    while current is not None:
        if current.incident_id.upper() == incident_id.upper():
            if verbose:
                print("\n[i] Ticket found:")
                print("-" * 70)
                print(f"{'Incident ID':<15}{current.incident_id}")
                print(f"{'Bot':<20}{current.bot}")
                print(f"{'Description':<40}{current.description}")
                print("-" * 70 + "\n")
            return current
        current = current.next

    if verbose:
        print(f"[!] Ticket {incident_id} not found.\n")
    return None
```

Explanation: The program starts at the head and checks each node's Incident ID. It compares the IDs without considering uppercase or lowercase letters. If a match is found, the ticket details are displayed. If no match is found, a "not found" message is shown.



Remove Resolved Incident:

```
def remove_ticket(self, incident_id):
    current = self.head
    previous = None

    while current is not None:
        if current.incident_id.upper() == incident_id.upper():
            if previous is None:
                self.head = current.next
            else:
                previous.next = current.next
            self.size -= 1
            print(f"[-] Ticket {incident_id} removed (resolved).\n")
            return True
        previous = current
        current = current.next

    print(f"[!] Ticket {incident_id} not found. Nothing removed.\n")
    return False
```

Explanation: The program uses current and previous pointers to find the ticket. If the ticket is the head, the head moves to the next node. Otherwise, previous.next skips the ticket being removed. The size is then decreased by 1.

Count Active Incidents:

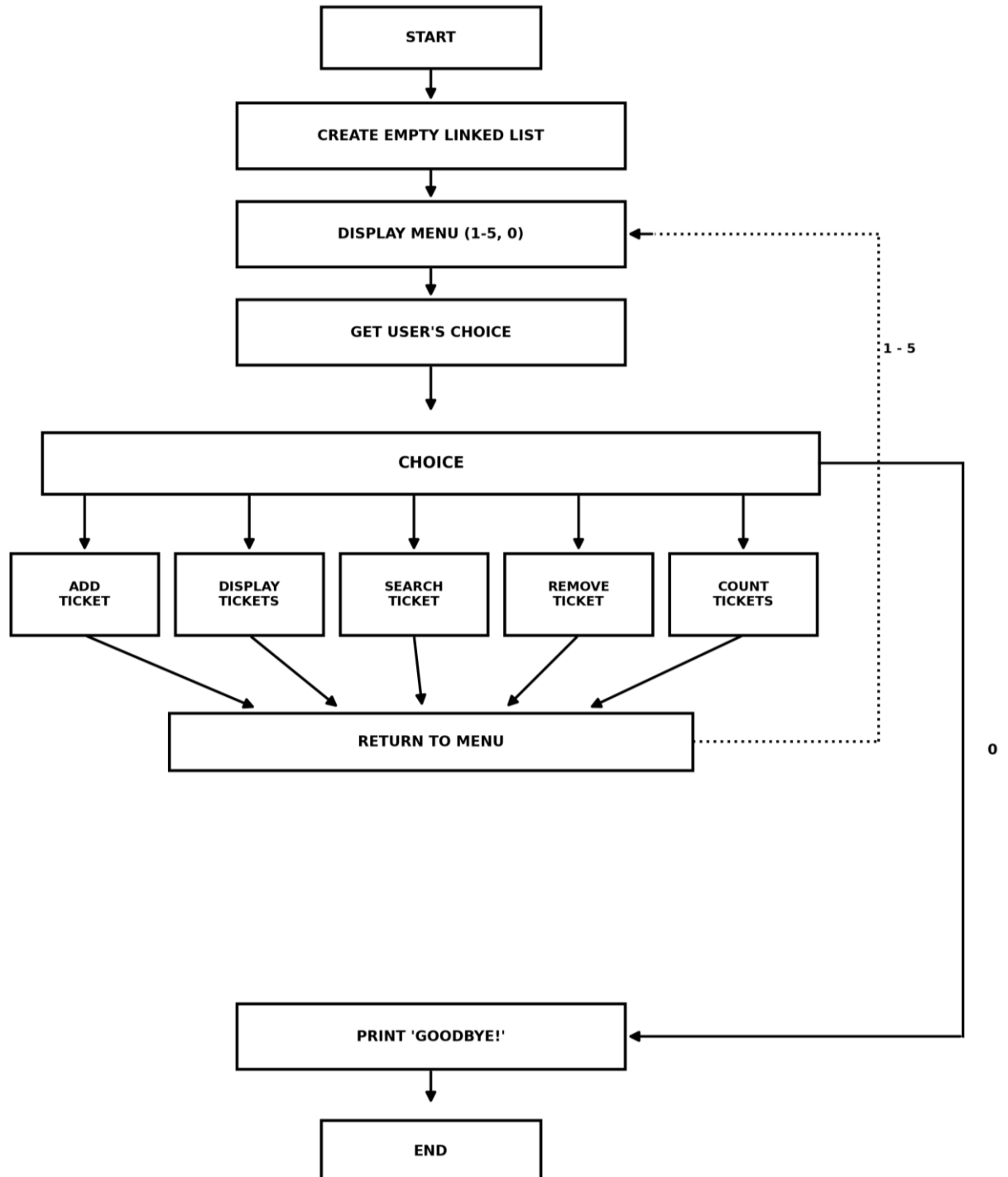
```
def count_active_tickets(self):
    print(f"[i] Total active incident tickets: {self.size}\n")
    return self.size
```

Explanation: The program displays and returns self.size, which keeps track of the number of active tickets. It increases when a ticket is added and decreases when a ticket is removed.



College of Computer Studies

III. **Flowchart:** Create a flowchart showing the overall process and logic of your program.





Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

- IV. **Conclusion:** Briefly explain what you learned from the activity and how the linear data structure was applied.

The overall things that I've learned in this activity is how the linked list works as a linear data structure in the code. I learned that each incident ticket is stored as a node and connected through pointers instead of being stored in fixed positions. I also learned how node traversal is used to add, display, search, and remove incident tickets. This activity helped me understand how a linked list can be used to organize and manage data in a simple way.